

PHP Backend Case Study — Orders & Pricing (Slim + Doctrine + PostgreSQL) — LIVE (60 minut)

Kontext

Stavíš část API pro order management fitness e-commerce. Slim 4, Doctrine ORM, PostgreSQL, UUID PK. DB je už migrovaná.

Tabulky (existují)

- customers(id, email, country, created_at)
- products(id, name, category, price)
- inventory(product_id, available)
- orders(id, customer_id, status, total_amount, created_at)
- order_items(id, order_id, product_id, quantity, unit_price)

Pozn.: Slevy budeme řešit **bez nové migrace** (uložíme snapshot slevy do **orders** jako jednoduché sloupce / nebo jen “in-memory” pro preview – viz Part 3). Cílem je neztrácet čas migracemi.

PART 1: Create Order (MUST) — 20–25 min

Endpoint: **POST** /api/orders

Vytvoří objednávku ve stavu **draft**.

Request

```
{
  "customer_id": "uuid",
  "items": [
    { "product_id": "uuid", "quantity": 2 }
  ]
}
```

Requirements (zjednodušené)

- **customer_id** musí být valid UUID string, jinak **422 INVALID_CUSTOMER_ID**
- customer musí existovat, jinak **404 CUSTOMER_NOT_FOUND**

- `items` musí existovat a mít aspoň 1 položku, jinak **422 EMPTY_ITEMS**
- `quantity >= 1`, jinak **422 INVALID_QUANTITY**
- Produkt musí existovat:
 - stačí vrátit **první** neplatné `product_id` (ne seznam), jinak **422 UNKNOWN_PRODUCT**
- Ulož:
 - `orders.id` nové UUID
 - `orders.status = draft`
 - `order_items.unit_price = products.price` v době vytvoření (snapshot)
- **Neřeš sklad** (až Part 2)

Response (201)

Vrátí order + items (stejně jako máš), `created_at` ISO8601.

Errors (Part 1)

- 400 `INVALID_JSON` (nevalidní JSON)
 - 422 `INVALID_CUSTOMER_ID`
 - 404 `CUSTOMER_NOT_FOUND`
 - 422 `EMPTY_ITEMS`
 - 422 `INVALID_QUANTITY`
 - 422 `UNKNOWN_PRODUCT`
-

PART 2: Confirm + Inventory (MUST, ale s menším scope) — 20 min

Endpoint: `POST /api/orders/{id}/confirm`

Requirements (zjednodušené)

- `{id}` musí být valid UUID, jinak **400 INVALID_ORDER_ID**
- order musí existovat, jinak **404 ORDER_NOT_FOUND**
- confirm jen ze stavu `draft`, jinak **409 ORDER_INVALID_STATE**

Confirm logika

V jedné DB transakci:

1. Načti order + items

2. Pro každý item ověř sklad:
 - pokud chybí řádek v `inventory` → **409 INVENTORY_MISSING** (stačí vrátit `product_id`)
 - pokud `available < quantity` → **409 OUT_OF_STOCK** (stačí vrátit `product_id + missing`)
3. Odečti sklad **race-condition safe**
Minimální požadavek: použij jeden z přístupů:
 - **SELECT ... FOR UPDATE** na `inventory` řádky, nebo
 - **UPDATE inventory SET available = available - :q WHERE product_id = :id AND available >= :q** a kontrola `rowCount()`
4. Spočti `total_amount = sum(quantity * unit_price)`
5. Nastav `status = confirmed` a ulož `total_amount`

Response (200)

```
{
  "order_id": "uuid",
  "status": "confirmed",
  "total_amount": 927.00
}
```

PART 3: Discount + Pricing preview (BONUS implementace, MUST je návrh) — 15 min

Endpoint: `POST /api/orders/{id}/apply-discount`

Zjednodušení: žádná tabulka `discount_codes`

Aby se neztrácel čas DB modelováním, discount kódy jsou hardcoded (v kódu) jako konfigurace:

- `WELCOME10` → percent 10
- `SAVE50` → fixed 50

Pozn.: Tohle pořád testuje normalizaci, validaci, pricing a snapshot, ale bez DB migrací a složité validity logiky.

Requirements

- `{id}` valid UUID → **400 INVALID_ORDER_ID**
- order existuje → **404 ORDER_NOT_FOUND**
- jen na `draft` → **409 ORDER_INVALID_STATE**
- request JSON `{ "code": "...", }`, jinak **400 INVALID_JSON**
- `code` musí být non-empty → **422 INVALID_DISCOUNT_CODE**
- normalizace: `trim + uppercase`
- pokud kód neexistuje → **404 DISCOUNT_NOT_FOUND**
- na jeden order max 1 sleva:
 - pokud už sleva existuje → **409 DISCOUNT_ALREADY_APPLIED**

Uložení slevy (zjednodušené)

Bez nové tabulky: uložit snapshot do `orders` (vhodné pro live):

- přidej do entity `Order` 3 nullable fieldy mapované na existující sloupce **jen pokud existují**; pokud neexistují, pak pouze “in-memory” a vrať preview bez ukládání.

Aby to bylo kompatibilní bez migrace, doporučuji toto pravidlo pro live:

- **MUST:** vrať pricing preview, uložení slevy je **BONUS** (kandidát řekne, jak by to persistnul).

Pricing definice

- `subtotal = sum(quantity * unit_price)`
- percent: `discount = round(subtotal * (value/100), 2)`
- fixed: `discount = value`
- `discount_total = min(subtotal, discount)`
- `total = subtotal - discount_total (min 0)`

Response (200)

```
{
  "order_id": "uuid",
  "status": "draft",
  "discount": { "code": "WELCOME10", "type": "percent", "value":
10.00 },
  "pricing_preview": { "subtotal": 927.00, "discount_total": 92.70,
"total": 834.30 }
}
```

Errors (Part 3)

- 400 `INVALID_ORDER_ID`
- 400 `INVALID_JSON`

- 404 ORDER_NOT_FOUND
 - 409 ORDER_INVALID_STATE
 - 422 INVALID_DISCOUNT_CODE
 - 404 DISCOUNT_NOT_FOUND
 - 409 DISCOUNT_ALREADY_APPLIED
-